Isfahan University of Technology
Faculty of Electrical and Computer Engineering

Application of Machine Learning Approaches to Classify the Adult Income Dataset

Statistical Pattern Recognition Course Project
M.Sc. in Computer Engineering - Artificial Intelligence and Robotics

Amirmohsen Sharifi

Course Instructor
Dr. Mohammadreza Ahmadzadeh

1402

Table of Contents

Abstract

This project examines the field of pattern recognition through the use of various machine learning algorithms on the Adult Income dataset. The objective is to determine, for a given input sample, whether a person's income is less than or greater than $50,000. To implement and compare the performance of distinct machine learning models, logistic regression, decision trees, Naive Bayes, XGBoost, and linear regression are used. The performance of these methods is compared both before feature selection and feature generation and after these operations. The initial phase involves careful data preprocessing to handle missing values and encode categorical features. Feature scaling is applied to ensure consistency across the dataset. This project investigates several feature selection techniques, such as principal component analysis, forward selection, and backward elimination, to determine their impact on model performance. Model evaluation uses a range of metrics, including accuracy, recall, F1 score, and mean squared error for linear regression. A custom decision tree implementation is developed by using information gain as an alternative to the Gini index. The project also investigates the application of Fisher linear discriminant analysis and examines feature selection through forward selection and backward elimination, analyzing their effects on model performance. It concludes with a comprehensive analysis of the results, highlighting the strengths and weaknesses of each algorithm.

Keywords

Statistical pattern recognition, machine learning, XGBoost, Naive Bayes, linear regression, logistic regression, decision tree, Fisher linear analysis, forward selection, backward elimination, principal component analysis (PCA)

Chapter 1

Introduction

1-1 Introduction

In the evolving field of data analysis, the application of machine learning enables the discovery of valuable patterns and predictions from complex datasets. This project undertakes the challenge of exploring a wide range of pattern recognition techniques while navigating the inherent complexities of real-world datasets. The dataset used in this project, obtained from Kaggle, reflects the complexities and nuances of real-world data environments [1]. Because external libraries are not permitted, the project follows a constraint that prohibits the use of pre-built machine learning tools, thereby requiring the initial implementation of various algorithms. A significant challenge encountered at the outset was the absence of a header in the dataset. This therefore required careful examination to decode the features and their significance. In addition, the dataset contains instances of missing values, requiring an appropriate approach to data imputation and handling. The strict restriction against using external libraries also made innovative solutions necessary for common machine learning tasks. For example, the lack of available tools led to custom implementations of logistic regression, decision tree, linear regression, and Naive Bayes. Handling missing data was another fundamental challenge during preprocessing. This project uses a combination of statistical imputation techniques, such as mean imputation for numerical features and mode imputation for categorical features, ensuring that subsequent machine learning models are trained on complete and representative data. As we proceed to the following sections, the project reveals a narrative of overcoming these challenges and emphasizes the importance of adaptability, ingenuity, and a comprehensive understanding of machine learning principles. The results obtained not only contribute to refining machine learning models but also provide insights into navigating the complexities of real-world data without relying on external libraries. The operation of the algorithms and a description of the project code are discussed in the following sections.

2-1 Logistic Regression

Logistic regression is a statistical method used for binary classification problems. Using the logistic function, it models the probability that a given sample belongs to a particular class. The logistic function, also known as the sigmoid function, maps any real-valued number to a range between 0 and 1. The parameters are optimized using methods such as maximum likelihood estimation to minimize logistic loss. Logistic regression is robust and interpretable and can be extended to handle multiple classes through techniques such as one-vs.-rest, which is widely used in fields such as medicine, finance, and the social sciences. Mathematically, the logistic regression model can be represented as follows:

$$P(Y=1)=\frac{1}{1+e^{-(b_0+b_1x_1+b_2x_2+\ldots+b_nx_n)}} \qquad (1-1)$$

In Equation $(1-1)$, the coefficients $b_n,\ldots,b_1,b_0$ are the optimized coefficients, and $x_n,\ldots,x_2,x_1$ are the input features. Optimization is usually performed using gradient descent or other optimization methods to minimize logistic loss. The trained model

can then be used to predict the probability that a sample belongs to the positive class, and a threshold can be applied for the final classification decision [2], [3].

3-1 Decision Tree

Decision trees are versatile machine learning models that perform well in both classification and regression tasks. The algorithm recursively partitions the dataset according to feature conditions and creates a tree structure in which each internal node represents a decision based on a feature, and each leaf node corresponds to the predicted outcome. Splitting criteria, often measured using Gini impurity or information gain, determine the feature on which the split should be made. Decision trees are interpretable, can capture complex decision boundaries, and can handle numerical and categorical data. However, they are prone to overfitting, which can be reduced through strategies such as pruning, limiting tree depth, or using ensemble methods.

The decision tree recursively partitions the dataset according to the feature that provides the best information gain or reduction in Gini impurity. The algorithm starts at the root node, selects the best feature on which to split, and creates child nodes accordingly. This process continues until a stopping criterion, such as reaching the maximum depth or having nodes with a minimum number of samples, is satisfied. The tree represents decision rules in a hierarchical structure, with each path from the root to a leaf representing a unique decision path. During prediction, an input sample traverses the tree from the root to a leaf, and the majority class at that leaf is the predicted class [4], [5].

4-1 Linear Regression

Linear regression is a fundamental algorithm for modeling the relationship between a dependent variable and one or more independent variables. It assumes a linear relationship in which the output is a weighted sum of the input features, often with an added bias term. The objective is to find optimal weights that minimize the sum of squared differences between predicted and actual values. Linear regression is interpretable and provides insight into feature importance and the direction of relationships. It is sensitive to outliers and assumes linearity; therefore, careful examination of the data characteristics is necessary. Regularization techniques such as Lasso or ridge regression can be used to prevent overfitting. The goal of linear regression is to find the line (for simple linear regression) or hyperplane (for multiple linear regression) that best fits the data. The model is represented as follows:

$$Y = b_0 + b_1 x_1 + b_2 x_2 + \ldots + b_n x_n \qquad (2-1)$$

In Equation $(2-1)$, the intercept is represented by b_0, and the optimized coefficients are represented by $b_n, \ldots, b_1$. Optimization is usually performed using the least-squares method, which minimizes the sum of squared differences between predicted and actual values. The resulting model provides coefficients that represent the contribution of each feature to the predicted output. During prediction, the input features are multiplied by their corresponding coefficients and summed to obtain the predicted output [6], [7].

XGBoost 5-1

XGBoost is an ensemble learning algorithm that has gained popularity because of its speed and predictive performance. It builds a strong predictive model by combining the outputs of multiple weak learners, often decision trees. XGBoost uses a gradient boosting framework in which each tree corrects the errors of previous trees. The model is optimized by minimizing a loss function that combines the errors and a regularization term. Hyperparameters such as tree depth, learning rate, and subsampling ratio affect model behavior. XGBoost handles missing data, is scalable, and performs well across different domains, making it a popular choice. Trees are added sequentially, and each new tree adjusts the predictions of the combined model. Regularization terms are introduced to prevent overfitting. XGBoost uses techniques such as pruning, column subsampling, and weighted quantile sketch to improve efficiency and speed [8], [9].[1][2][3]

6-1 Naive Bayes

Naive Bayes is a probabilistic classification algorithm based on Bayes' theorem. Despite its simplifying assumption of feature independence, Naive Bayes often achieves strong performance. The algorithm calculates the posterior probability of each class given a set of features, assuming that the features are conditionally independent. Naive Bayes is computationally efficient and requires a relatively small amount of training data for parameter estimation. It is particularly suitable for high-dimensional datasets, and its efficiency makes it a popular choice in situations where real-time prediction is essential. Although the independence assumption may not always hold, Naive Bayes can still provide competitive results. Naive Bayes is a probabilistic algorithm based on Bayes' theorem. Given a set of features $X = x_1, x_2, \ldots, x_n$, the probability of class C according to Bayes' theorem is given by the following equation:

$$P(C|X) = \frac{P(X|C)P(C)}{P(X)} \quad (3-1)$$

In Equation $(3-1)$, $P(C|X)$ is the posterior probability, $P(X|C)$ denotes the likelihood, and $P(C)$ denotes the prior probability, while P(X) is the evidence. The assumption is that the features are independent given the class, which simplifies the computation. During training, class and feature probabilities are estimated from the training data. During prediction, the class with the highest posterior probability is selected [10].[4]

7-1 Algorithm Description

First, the required libraries must be imported. In this project, the numpy,pandas,matplotlib libraries are used. The file is then read using the pandas library. Next, we distinguish the target and non-target features. To obtain good machine learning model performance, the missing values in the data must be handled. Missing numerical values are replaced with the

[1] Pruning
[2] Column Subsampling
[3] weighted quantile sketch
[4] Evidence

mean, and missing categorical values are replaced with the mode. The data for the target and non-target features must then be encoded. The dataset is subsequently divided into training and test sets. In this project, 80% is allocated to training and 20% to the test data. Then, to prevent a substantial reduction in machine learning model accuracy, the non-target data must be scaled. In this project, min-max scaling is used.

To use XGBoost, we converted the data to the DMatrix format (the internal XGBoost data structure). We then specified the XGBoost hyperparameters. We also used the early stopping technique to prevent overfitting. early stopping is a technique used in machine learning to prevent a model from overfitting the training data. Overfitting occurs when a model learns the training data, including its noise and fluctuations, too well, to the extent that it performs poorly on new and unseen data. early stopping helps address this problem by monitoring model performance on a validation dataset during training and stopping the training process when validation performance begins to decline.

We then implemented logistic regression according to the theoretical concepts discussed above, measured its accuracy on the training set, and executed it to make predictions on the test data, storing and reporting its accuracy. The precise implementation is as follows: (logistic_regression_train) is used for binary classification. This function takes the input features and labels, initializes the weights to zero, and repeatedly updates them using gradient descent for a specified number of epochs. Within the loop, it computes predictions, errors, and gradients and updates the weights accordingly. In addition, it tracks training accuracy during each epoch. After training, the function returns the final weights and a list of training accuracies. The training is then applied to the dataset (x_train_scaled and y_train). Subsequently, the trained weights are used to make predictions on the test set (x_test_scaled), and accuracy is evaluated by comparing the predicted labels with the actual labels (y_test).

We then implemented linear regression according to the theoretical concepts discussed above, measured its accuracy on the training set, and executed it to make predictions on the test data, storing and reporting its accuracy. The precise implementation is as follows: the linear_regression_train function takes the input features and labels, initializes the weights and biases to zero, and repeatedly updates them using gradient descent for a specified number of epochs. Within the loop, it computes predictions, errors, and gradients for the weights and biases and adjusts them to minimize the mean squared loss. The mean squared loss for each epoch is added to a list. After training, the function returns the final weights, bias values, and a list of the mean squared losses during training. The training is then applied to the dataset (x_train_scaled and y_train). Subsequently, the biases and trained parameters are used to make predictions on the test set (x_test_scaled). The mean squared loss on the test set is calculated by comparing the predicted values with the actual labels (y_test).

We then implemented the decision tree according to the theoretical concepts discussed above, measured its accuracy on the training set, and executed it to make predictions on the test data, storing and reporting its accuracy. The precise implementation is as follows: the

Node class represents a node in the decision tree, with features such as feature_index and a threshold for splitting the data, left and right for the child nodes, and value for storing the predicted class label. The DecisionTree class is initialized with an optional max_depth parameter indicating the maximum depth the tree can reach. The fit method trains the decision tree by constructing it recursively using the _build_tree method. The tree-building logic involves finding the best split using the _find_best_split method. The _predict_tree method is responsible for recursively predicting the class labels of samples based on the trained tree. In practice, after loading and preprocessing the dataset, an instance of the DecisionTree class is created with the specified max_depth. The tree is then trained on the training set (x_train_scaled and y_train). Predictions are made on the test set (x_test_scaled) using the prediction method, and accuracy is evaluated by comparing the predicted labels with the actual labels (y_test).

We then implemented Naive Bayes according to the theoretical concepts discussed above, measured its accuracy on the training set, and executed it to make predictions on the test data, storing and reporting its accuracy. The precise implementation is as follows: during initialization, methods for initialization, fitting, and prediction set the class_probs and feature_probs attributes to None. The fit method is responsible for training the Naive Bayes classifier using the input training data (X_train and y_train). It calculates class probabilities by counting the occurrences of each class and normalizing by the total number of samples. In addition, it calculates feature probabilities for each class and uses Laplace smoothing to avoid zero probabilities. The probabilities are stored in class_probs and feature_probs for later use during prediction. The prediction method uses the trained probabilities to make predictions on a given test set (X_test). For each sample in the test set, it calculates the posterior probability of each class using the Naive Bayes relationship, taking the class and feature probabilities into account. The class with the highest posterior probability is assigned as the predicted class for that sample. Predictions for all test samples are collected and returned as a NumPy array. In essence, the Naive Bayes classifier in this code relies on the assumption of independence among the features given the class, making it computationally efficient and suitable for various classification tasks, particularly those involving discrete features.

After examining the dataset using the machine learning methods without feature selection or feature generation, we now separately apply PCA to the dataset again, retaining 95% of the variance, and provide the resulting data to the designed machine learning algorithms. The PCA section operates as follows. First, the code computes the covariance matrix (cov_matrix) of the scaled training data (x_train_scaled) using the NumPy np.cov function, with the rowvar=False parameter indicating that each column represents a variable and each row represents an observation. The corresponding eigenvalues and eigenvectors of the covariance matrix are then computed using the NumPy np.linalg.eigh function. The eigenvalues represent the variance along the principal components, while the eigenvectors represent the directions of these principal components in feature space. After

eigendecomposition, the code sorts the eigenvalues and their corresponding eigenvectors in descending order. The goal is to prioritize the principal components that contribute most to the overall variance of the dataset. To achieve dimensionality reduction while retaining a specified percentage of the total variance (95% in this case), the code selects a subset of the sorted eigenvalues and their associated eigenvectors. Finally, the data are projected onto the selected principal components, effectively transforming the original high-dimensional dataset into a lower-dimensional space. While the code provides a concise outline of PCA implementation, PCA fundamentally works by identifying directions (principal components) in the data space that capture maximum variance, enabling dimensionality reduction while preserving as much information as possible. The resulting x_train_pca and x_test_pca are then provided to the machine learning algorithms we designed, and the accuracy is measured on the new dataset.

We now separately execute forward selection and backward elimination on the dataset. For forward selection, the process repeatedly evaluates potential features for inclusion in a model with the goal of maximizing classification accuracy. The function takes a feature matrix and the corresponding labels as input. During each iteration, it identifies the feature to be considered. This process continues for a predetermined number of iterations, limited by the total number of features or a specified maximum. The algorithm maintains two sets: Selected_features, which collects the features selected for inclusion in the model, and the remaining features, which include all features that have not yet been considered. At each stage, the code evaluates the potential contribution of each remaining feature and selects the one that results in the greatest improvement in accuracy. The selected feature is then added to the model and the process is repeated. The final result is a list of features that collectively optimizes the accuracy of a logistic regression model.

Backward elimination works by encapsulating a Backward Elimination feature selection method. The objective of this iterative algorithm is to identify and discard the least informative features in order to optimize the performance of a logistic regression model. Given a feature matrix and the corresponding labels, the process begins by including all features in select_features. At each iteration, it evaluates the effect of removing each feature and calculates the accuracy of the trained logistic regression model on the resulting feature subset. The algorithm identifies the feature whose removal causes the smallest decrease in accuracy, and that feature is subsequently removed from the selected set. This process is repeated until the optimal feature subset is identified. The result is a list of features that generally improves the accuracy of the logistic regression model. The x_train_selected and x_test_selected are then provided to the machine learning algorithms we designed.

8-1 Results

Overall, the results were obtained for the five algorithms XGBoost, logistic regression, linear regression, decision tree, and Naive Bayes in three stages. In Stage 1, without performing PCA, forward selection, or backward elimination, only the data are scaled independently and provided to the machine learning algorithms in the form of the variables x_train_scaled and x_test_selected, and the algorithms are executed. In Stage 2, the data are

independently processed again, excluding Stage 1 and using only PCA, and are provided to the machine learning algorithms in the form of x_train_pca and x_test_pca. In Stage 3, the data are again processed independently, excluding Stages 1 and 2, and are passed through the forward selection and backward elimination algorithms in the form of x_train_selected and x_test_selected before being provided to the machine learning algorithms. The results obtained are presented below.

1-8-1 Stage 1

```
Accuracy: 0.8738867847271983
```

Figure 1-1: XGBoost Accuracy in Stage 1

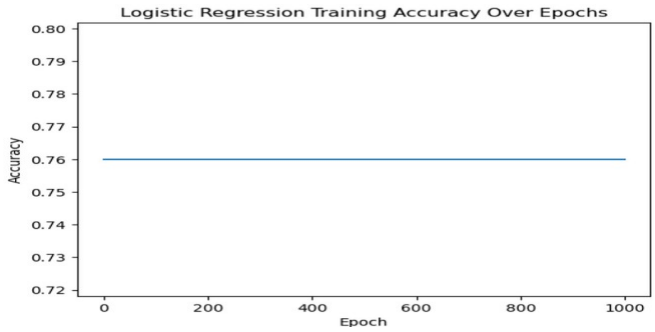

Figure 1-2: Logistic Regression Accuracy in Stage 1

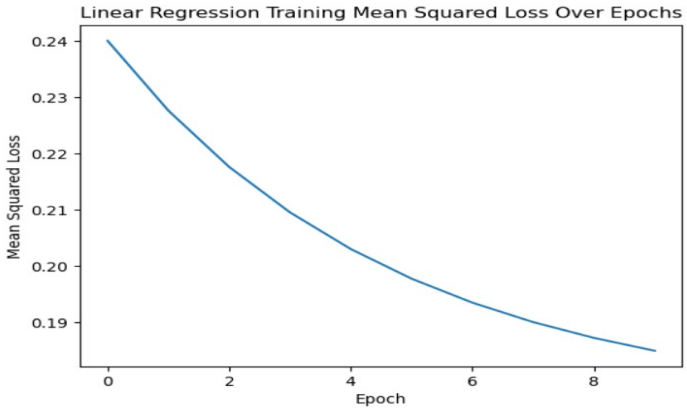

Figure 1-3: Linear Regression Error in Terms of Mean Squared Error in Stage 1

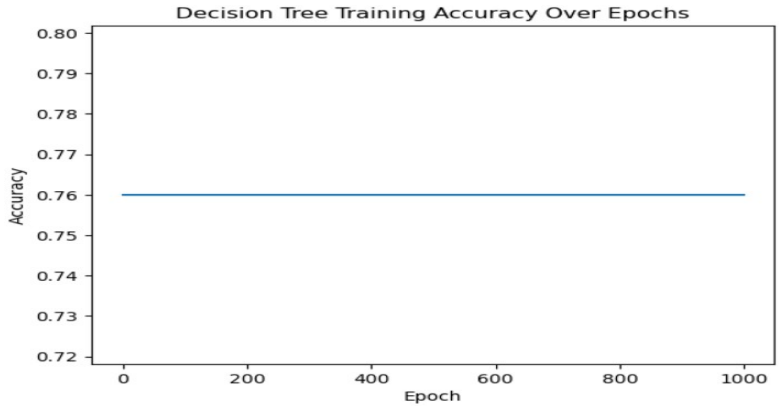

Figure 1-4: Decision Tree Accuracy in Stage 1

```
Naive Bayes Test Accuracy: 0.6205343433309448
```

Figure 1-5: Naive Bayes Accuracy in Stage 1

2-8-1 Stage 2

```
Test Accuracy: 0.8541304125294298
```

Figure 1-6: XGBoost Accuracy in Stage 2

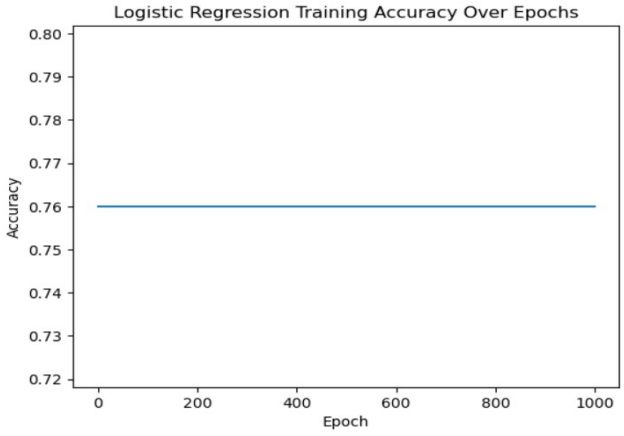

Figure 1-7: Logistic Regression Accuracy in Stage 2

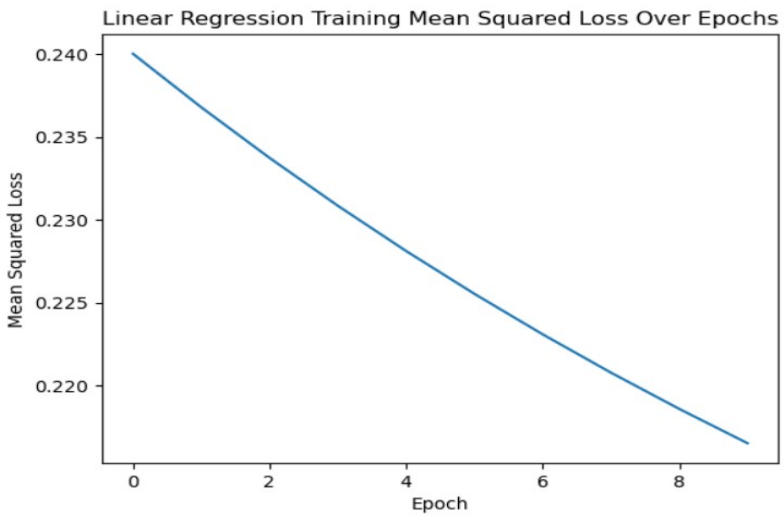

Figure 1-8: Linear Regression Error in Terms of Mean Squared Error in Stage 2

Figure 1-9: Decision Tree Accuracy in Stage 2

```
Naive Bayes Test Accuracy: 0.6091718702016583
```

Figure 1-10: Naive Bayes Accuracy in Stage 2

3-8-1 Stage 3

```
Test Accuracy: 0.8334527587265841
```

Figure 1-11: XGBoost Accuracy in Stage 3

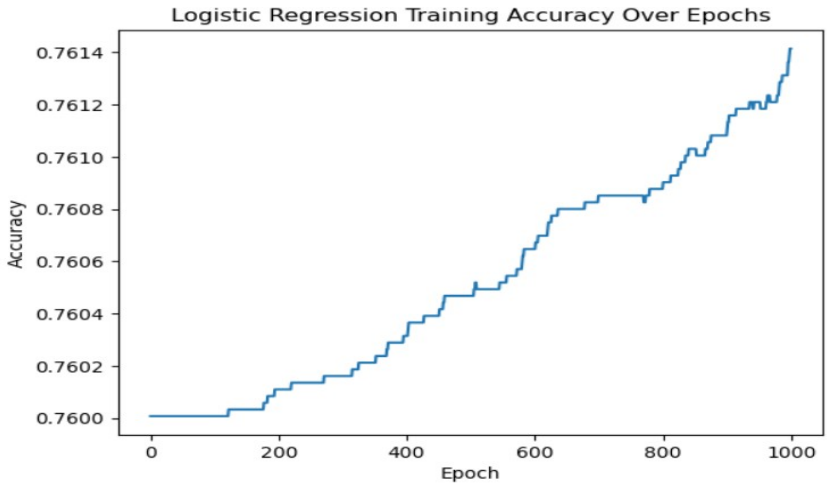

Figure 1-12: Logistic Regression Accuracy in Stage 3

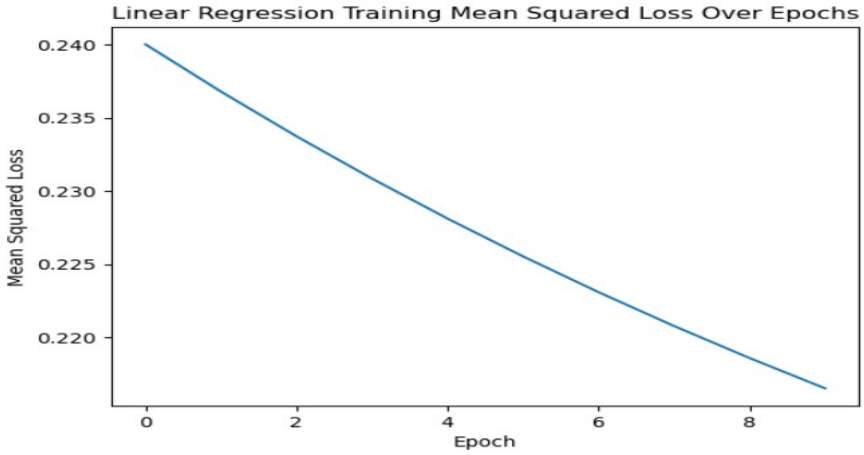

Figure 1-13: Linear Regression Error in Terms of Mean Squared Error in Stage 3

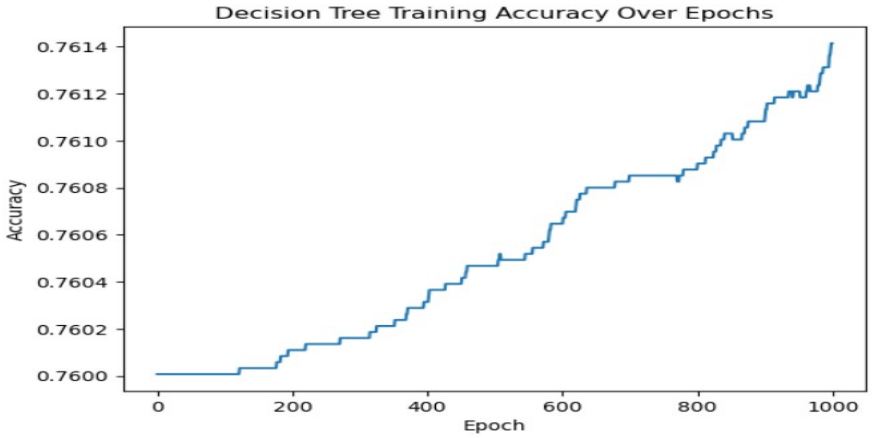

Figure 1-14: Decision Tree Accuracy in Stage 3

```
Naive Bayes Test Accuracy: 0.6091718702016583
```

Figure 1-15: Naive Bayes Accuracy in Stage 3

9-1 References

[1] F. Ding, M. Hardt, J. Miller, and L. Schmidt, “Retiring Adult: New Datasets for Fair Machine Learning,” *Adv. Neural Inf. Process. Syst.*, vol. 8, no. NeurIPS, pp. 6478–6490, 2021.

[2] J. Daniel and J. H. Martin, “Part1_Ch5. Logistic Regression,” *Speech Lang. Process.*, 2023.

[3] C. Y. J. Peng, K. L. Lee, and G. M. Ingersoll, “An introduction to logistic regression analysis and reporting,” *J. Educ. Res.*, vol. 96, no. 1, pp. 3–14, 2002, doi: 10.1080/00220670209598786.

[4] Y. Y. Song and Y. Lu, “Decision tree methods: applications for classification and prediction,” *Shanghai Arch. Psychiatry*, vol. 27, no. 2, pp. 130–135, 2015, doi: 10.11919/j.issn.1002-0829.215044.

[5] J. R. Quinlan, “Induction of decision trees,” *Mach. Learn.*, vol. 1, no. 1, pp. 81–106, 1986, doi: 10.1007/bf00116251.

[6] M. Statistics, “Linear Regression,” pp. 1–21, 2001.

[7] K. Kumari and S. Yadav, “Linear regression analysis study,” *J. Pract. Cardiovasc. Sci.*, vol. 4, no. 1, p. 33, 2018, doi: 10.4103/jpcs.jpcs_8_18.

[8] C. Bentéjac, A. Csörgő, and G. Martínez-Muñoz, “A Comparative Analysis of XGBoost,” no. February, 2019, doi: 10.1007/s10462-020-09896-5.

[9] Z. Arif Ali, Z. H. Abduljabbar, H. A. Tahir, A. Bibo Sallow, and S. M. Almufti, “eXtreme Gradient Boosting Algorithm with Machine Learning: a Review,” *Acad. J. Nawroz Univ.*, vol. 12, no. 2, pp. 320–334, 2023, doi: 10.25007/ajnu.v12n2a1612.

[10] Rish I, “An empirical study of the naive bayes classifier,” *IJCAI 2001 Work. Empir. methods Artif. Intell.*, no. January, pp. 41–46, 2001, [Online]. Available: http://www.cc.gatech.edu/home/isbell/classes/reading/papers/Rish.pdf